

WS-REV-001

Workstream Review and Revision Lifecycle

v0.1 Specification

The definitive v0.1 contract for centralized reviewer authorization, immutable artifact-backed judgment, preferred-to-open routing, claim leases, revision replay, finding evidence, and terminal decisions.

STATUS	Approved design baseline for implementation
MATURITY	Design complete; not yet runtime-proven
BOUNDARY	Authorization · Artifact evidence · Review · Revision
DATE	15 July 2026

Contents

This document defines the complete Workstream v0.1 review and revision boundary, including its Authorization Service and Artifact Storage integration contracts. Section numbering follows the approved Markdown source.

Contents	2
1. Purpose	4
2. Implementation Status and Proof Boundary	4
3. Locked v0.1 Decisions	5
4. Actor and Authority Model	6
5. Core Object Model	11
6. Project Review Policy and Artifact Storage Contract	19
7. Queue Views and Ordering	23
8. Reviewer Eligibility	24
9. State Machines	24
10. Timer Semantics	25
11. Core Operations	26
12. Decision Contracts	29
13. Revision Routing	31
14. Grant Revocation During an Active Lease	32
15. API Contract	32
16. Error Contract	34
17. Database Constraints and Indexes	35
18. Concurrency and Transaction Requirements	36
19. Background Processing and Recovery	38
20. Audit Events	39
21. Notifications and Operational Views	41
22. Observability	42
23. Security Requirements	43

24. End-to-End Reference Sequences	43
25. Conformance Tests	45
26. Implementation Delivery Order	48
27. Definition of Done	50
28. Explicitly Out of Scope	51
29. Future Additive Directions	51
30. Coding-Agent Handoff Rules	52

1. Purpose

This specification defines what Workstream does after a finalized submission passes the durable post-submission checker gate and becomes ready for human review.

It establishes:

- who may submit and review work;
- how reviewer authority is granted;
- which exact Authorization Service permissions and resource contexts guard every review and revision operation;
- how first submissions enter an open review pool;
- how revisions return to the previous reviewer as a preferred backlog;
- how preferred work falls back into an open FIFO pool;
- how a reviewer claims exactly one review at a time through a lease;
- how expired, released, or revoked leases return work safely to the queue;
- how submission versions, reviews, findings, and finding resolutions remain immutable;
- how `accept`, `needs_revision`, and `reject` affect the task and submitter assignment;
- how an authorized reviewer consumes the exact immutable submission and checker artifacts under review;
- how optional finding evidence is verified, bound, retained, and projected without moving Review authority into storage;
- which events must be auditable now so future reputation policy can use them later.

The design is informed by real high-volume contribution and review operations in which the global queue may be empty while individual reviewers still have revision backlogs routed to them.

2. Implementation Status and Proof Boundary

The coding agent MUST preserve the difference between proven behaviour and locked design.

Lifecycle area	Status
Guide source snapshot, sufficiency, and policy derivation	Proven
Manager approval of an exact submission policy	Proven
Effective policy and deterministic checker compilation	Proven
Task policy-context locking	Proven
Contributor claim, pre-submit, and submission finalization	Proven
Durable post-submission checker run	Proven
Transition to <code>review_pending</code>	Proven
Project reviewer grants	Locked by this specification
ActorProfile and centralized Authorization Service contract	Locked by <code>WS-AUTH-001</code> ; implementation proof required before this lifecycle

Lifecycle area	Status
Review artifact retrieval and finding-evidence binding	Locked by this revision; not yet runtime-proven
Review snapshot projection to Artifact Storage/search	Locked by this revision; not yet runtime-proven
Preferred-reviewer backlog and open FIFO pool	Locked by this specification
Claim leases and capacity enforcement	Locked by this specification
Immutable review and revision chain	Locked by this specification
Reject and submitter-task blocking	Locked by this specification
Reviewer/submitter contribution and compensation boundary	Specified by WS-CON-001; implementation integration remains to be proven
Reputation policy	Deferred

The implementation begins at this precondition:

```
Submission finalized
-> SubmissionVersion created
-> all required submission ArtifactBindings are verified and retained
-> task enters evaluation_pending
-> durable checker completes
-> routing_recommendation = allow_review
-> task enters review_pending
-> ReviewQueueEntry is created
```

A checker failure is not a human [Review](#) and MUST NOT be stored as [accept](#), [needs_revision](#), or [reject](#). Automated evaluation remains upstream of this specification.

3. Locked v0.1 Decisions

The following decisions are part of this contract and are not left to coding-agent interpretation.

1. Workstream has two major actor authority domains: Admin and Contributor.
2. Contributor project roles are [submitter](#), [reviewer](#), or [both](#).
3. The v0.1 Admin roles are Access Administrator, Operator, Project Manager, Finance Authority, and Audit Authority as defined by WS-AUTH-001.
4. Admin authority does not automatically grant reviewer authority.
5. An Admin who reviews work MUST also hold an active contributor-side reviewer grant for that project.
6. Skills and reputation inform reviewer-grant decisions but do not automatically grant reviewer authority in v0.1.
7. External reviewers use the same contributor and role-grant model after identity onboarding.
8. A contributor cannot review their own submission in v0.1.
9. A reviewer may hold exactly one active review lease globally in v0.1.
10. First submission versions enter the open FIFO review pool.
11. Revision submission versions return to the reviewer who issued the prior [needs_revision](#) decision for a time-limited preferred window.

12. A preferred-reviewer backlog and the open FIFO pool are distinct routing views over the same queue-entry model.
13. Preference time and review-lease time are separate timers.
14. When preference expires without a claim, the entry becomes open while preserving its original queue age.
15. When a claimed lease expires or is manually released, reviewer stickiness is cleared and the entry returns to the open pool.
16. Queue routing records are mutable; submission versions, reviews, findings, finding resolutions, and completed lease attempts are permanent.
17. `accept`, `needs_revision`, and `reject` are the only human review decisions.
18. `reject` is terminal for the task in v0.1.
19. Reject blocks the submitter from that task only; it does not revoke their project membership or project role grant.
20. Every valid committed Review is a reviewer contribution regardless of `accept`, `needs_revision`, or `reject`; `accept` additionally recognizes the submitter contribution. `WS-CON-001` owns those records and compensation effects.
21. `WS-AUTH-001` is the sole authority for ActorProfile resolution, grants, permission evaluation, resource scope, and authorization decisions.
22. Review routers and lifecycle services MUST NOT query role tables or rebuild permission logic locally.
23. Initial and revised submissions use the same `submission.create` permission and submission operation; no `submission.revise` permission exists.
24. Workstream remains the authority for queues, leases, Reviews, findings, resolutions, tasks, and assignments.
25. Artifact Storage owns artifact bytes, integrity receipts, manifests, retention, and optional search projections; it never owns a review decision or lifecycle transition.
26. Reviewers consume submission and checker evidence only through Workstream-authorized Artifact Storage access.
27. Human Identity Issuer tokens are never forwarded to Artifact Storage. Workstream uses service identity and a least-privilege storage scope after authorizing the human request.
28. Public review and finding contracts reference Workstream `ArtifactBinding` IDs, never unverified paths, arbitrary URLs, or caller-supplied provider CIDs.
29. Required evidence unavailability blocks review completion but never becomes `reject`, `needs_revision`, a checker failure, or contributor fault.
30. Review and finding snapshot projection is asynchronous and retryable. Projection failure never rolls back or mutates a committed canonical Review.

4. Actor and Authority Model

4.1 Major actor domains

```

Workstream Actor
├── Admin
│   ├── Project Manager
│   ├── Operator
│   ├── Finance Authority
│   ├── Audit Authority
│   └── Access Administrator
├── Contributor
├── Submitter
├── Reviewer
└── Both

```

These are authority domains, not separate profiles. One Identity Issuer subject resolves to one Workstream ActorProfile. That ActorProfile may participate in the Contributor domain by default and may separately hold AdminRoleGrant and ProjectRoleGrant records. Each effective authority must still be explicitly granted; one domain never implies the other.

4.2 Identity boundary

Identity Issuer establishes the external identity anchor. **WS-AUTH-001** resolves that identity to one Workstream ActorProfile and performs every Workstream authorization decision.

```

Identity Issuer token
-> TokenVerifier verifies issuer, subject, audience, validity, and coarse scopes
-> AuthorizationService builds the current AuthorizationContext
-> Workstream resolves the canonical project and resource relationships
-> AuthorizationService evaluates the registered permission and resource guards
-> lifecycle service evaluates queue, lease, review, revision, and artifact-state
invariants
-> command/query proceeds only when both layers allow it

```

A valid identity token never proves that the actor is a submitter, reviewer, Project Manager, or Operator.

The Identity Issuer token is consumed at the Workstream boundary. It **MUST NOT** be forwarded to the configured Artifact Storage provider.

4.3 Admin authority relevant to this specification

Action	Minimum Workstream admin authority
Issue or revoke a contributor project role grant	Project Manager whose scope covers the project
View all project review queues and reviewer backlogs	Project Manager or Operator
Clear a preferred-reviewer route	Project Manager or Operator
Force-release an active review lease	Operator
Close a queue entry administratively	Operator
Correct routing metadata through a compensating event	Operator

Finance and Audit Authorities receive no review-routing mutation privilege solely from those roles. Audit Authority may inspect review records only through its separately granted read permissions and covered scope.

4.4 Contributor authority

Contributor grant	May claim submission tasks	May claim review work
submitter	Yes	No
reviewer	No	Yes
both	Yes	Yes, except own submissions

4.5 Normative relationship to WS-AUTH-001

WS-AUTH-001 supersedes the authority portions of this document wherever ActorProfile, Identity Issuer verification, AdminRoleGrant, ProjectRoleGrant, permission assignment, scope resolution, actor suspension, identity-link revocation, or authorization-decision behavior is concerned.

This specification remains authoritative for:

- review queue routing and ordering;
- claim and lease invariants;
- self-review and current-lease resource guards;
- immutable Review, ReviewFinding, FindingResolution, and revision-chain behavior;
- decision-specific task and assignment transitions;
- review-evidence availability and ArtifactBinding guards.

Every protected request MUST first obtain an AuthorizationDecision. An allowed decision means only that the actor may attempt the operation. It does not mean that the lifecycle operation is valid in the resource's current state.

```
AuthorizationService
-> may this ActorProfile attempt this permission on this canonical project resource?

Review/Revision domain service
-> does the queue, lease, submission, artifact, and task state permit it now?
```

4.6 Required permission mapping

The following permission identifiers are consumed from WS-AUTH-001.

Lifecycle operation	Required permission	Additional lifecycle guard
Read an approved project	<code>project.read</code>	Actor has effective project scope
Submit an initial or revision version	<code>submission.create</code>	Own active TaskAssignment; task permits submission; not blocked
Read own submission/review chain	<code>submission.read_own</code> and <code>review.chain.read</code>	Actor owns the task submission chain
Read next/current reviewer work	<code>review.queue.read</code>	Reviewer/both grant; eligible project; one-active-lease rule

Lifecycle operation	Required permission	Additional lifecycle guard
Inspect full project queue operationally	<code>review.queue.inspect</code>	Covered Project Manager, Operator, or Audit scope
Claim offered review	<code>review.claim</code>	Pending entry; preferred routing; no self-review; no active lease
Release owned review lease	<code>review.release</code>	Actor owns active lease
Decline unclaimed preference	<code>review.decline_preference</code>	Actor is current preferred reviewer
Read submission artifacts for claimed review	<code>submission.read_for_review</code>	Actor owns active lease for that submission version
Read prior review/revision chain	<code>review.chain.read</code>	Own submission chain, owned review lease, or covered admin/audit scope
Record review decision and findings	<code>review.decision</code>	Active unexpired owned lease; current reviewer grant; evidence available
Force-release lease	<code>review.lease.force_release</code>	Covered Operator authority and mandatory reason
Assign or clear preferred routing	<code>review.queue.override</code>	Covered Operator or Project Manager authority and mandatory reason
Administratively close queue entry	<code>review.queue.override</code>	Covered Operator authority and mandatory reason
Run expiry sweep	<code>operations.timer.run</code>	Provisioned Workstream system actor
Run review reconciliation	<code>operations.reconcile.run</code>	Provisioned Workstream system actor
Execute normal artifact/projection outbox retry	<code>operations.outbox.retry</code>	Provisioned Workstream system actor
Request recovery of a dead-lettered/uncertain artifact operation	<code>artifact_recovery.request</code>	Covered Operator authority; worker executes the recovery
Rebuild review search projection	<code>operations.projection.rebuild</code>	Provisioned Workstream system actor or authorized Operator

The following additive permissions are required if they are not yet present in the implemented WS-AUTH-001 catalogue:

- `review.queue.override` covers direct preferred routing, clearing a preferred route, and administrative queue closure. It does not grant review-decision authority or lease force-release authority.

- `artifact_recovery.request` lets an authorized Operator request recovery of an uncertain/dead-lettered artifact operation. It never makes the Operator the execution-lease owner; the system worker executes and fences recovery according to the Artifact Storage recovery contract.

There is no `submission.revise`, `review_revision.record`, or `review.assign` contributor permission in v0.1.

4.7 Canonical resource contexts

The authorization service **MUST** receive resource relationships resolved from Workstream records, never trusted from request JSON.

Resource	Canonical project resolution	Required guard attributes
ProjectRoleGrant	<code>ProjectRoleGrant.project_id</code>	target contributor; issuer; status; role
SubmissionVersion	<code>SubmissionVersion.task_id -> Task.project_id</code>	creator; task assignment; version; task state
ReviewQueueEntry	stored <code>project_id</code> , cross-checked through Task and SubmissionVersion	queue state; routing mode; preferred reviewer; submission creator
ReviewLease	<code>Lease -> QueueEntry -> Project</code>	reviewer owner; status; expiry; active queue reference
Review / review chain	<code>Review -> SubmissionVersion -> Task -> Project</code>	reviewer; submitter; immutable chain relationships
ArtifactBinding	Binding owner -> SubmissionVersion, CheckerRun, ReviewFinding, or FindingResponse -> Task -> Project	artifact role; visibility; integrity state; owning lifecycle record

Any mismatch across these ownership paths fails closed with `resource_project_mismatch` and is also an integrity alert.

4.8 Authorization evaluation points

Authorization **MUST** be evaluated:

1. before returning queue, submission, review-chain, or artifact metadata;
2. inside the same application transaction used to claim a review;
3. inside the same application transaction used to release or decline work;
4. inside the decision transaction before Review creation;
5. again before a worker commits an actor-attributed deferred command;
6. at every artifact retrieval authorization, even if the actor previously opened the Review Context.

Claim and decision operations **MUST NOT** rely only on a router-level decision made before their transaction. Revocation, suspension, or grant replacement may occur between request parsing and commit.

4.9 AuthorizationDecision consumption

For sensitive review mutations, the lifecycle records or links the `WS-AUTH-001` decision containing at least:

```
AuthorizationDecisionLink:
decision_id: uuid
actor_profile_id: uuid
permission: string
resource_type: string
resource_id: uuid
resolved_project_id: uuid
allowed: bool
effective_grant_id: uuid | null
denial_code: string | null
policy_version: string
decided_at: timestamp
```

The lifecycle stores the decision reference and approved non-secret fields required for audit. It never stores the raw token or treats a prior allowed decision as reusable authorization for a later request.

5. Core Object Model

5.1 ProjectRoleGrant

ProjectRoleGrant is the source of contributor authority within one project. Its canonical schema, creation authority, scope, replacement, revocation, and authorization behavior are owned by **WS-AUTH-001**. The representation below is the subset consumed by the review lifecycle and **MUST NOT** be implemented as a second grant model.

```
ProjectRoleGrant:
id: uuid
project_id: uuid
contributor_id: uuid
role: enum [submitter, reviewer, both]
status: enum [active, revoked]

grant_method: enum [manual, automated]
automation_policy_ref: uuid | null

qualification_snapshot_ref: uuid

granted_by: actor_id
granted_at: timestamp
revoked_by: actor_id | null
revoked_at: timestamp | null
revoked_reason: string | null
```

Qualification snapshot

qualification_snapshot_ref points to an immutable snapshot created when the grant is issued. It records what the Admin could evaluate at grant time without requiring the future reputation engine to exist.

```
ProjectRoleQualificationSnapshot:
id: uuid
project_id: uuid
contributor_id: uuid
skills_snapshot: object
reputation_snapshot: object
prior_project_work_refs: list[uuid]
external_expertise_refs: list[string]
captured_at: timestamp
captured_by: actor_id
```

An unavailable reputation value is recorded explicitly as unavailable; it does not prevent a manual grant.

ProjectRoleGrant invariants

- A submitter claim requires an active grant with `role IN (submitter, both)` for the same project.
- A review claim requires an active grant with `role IN (reviewer, both)` for the same project.
- A review decision re-checks the same active grant at decision time.
- Grants are project-scoped.
- `grant_method = manual` is the only creation path enabled in v0.1.
- `grant_method = automated` is schema-reserved but MUST NOT be emitted by v0.1 code.
- Claim-time authority treats manual and future automated grants identically.
- Grant revocation never deletes historical reviews or lease attempts.

5.2 SubmissionVersion

SubmissionVersion is the immutable work version being evaluated.

```
SubmissionVersion:
id: uuid
task_id: uuid
version_number: int
submission_id: uuid
artifact_hash: string
prior_submission_version_id: uuid | null
created_by: contributor_id
created_at: timestamp
```

SubmissionVersion invariants

- Immutable from creation.
- `version_number` is strictly increasing per task and starts at 1.
- `prior_submission_version_id` is null only for version 1.
- Every later version points to the immediately preceding version for the same task.
- The creator is the submitter whose task assignment is being evaluated.
- The creator cannot claim the corresponding review queue entry.
- At most one canonical `Review` may exist for a submission version.
- A version whose queue entry closes without `review_recorded` may have no Review; therefore the database invariant is at most one, not unconditionally exactly one.

5.3 SubmissionFindingResponse

When a contributor creates a revision version, the submission MUST respond to every unresolved blocking finding from the prior review.

```
SubmissionFindingResponse:
  id: uuid
  submission_version_id: uuid
  finding_id: uuid
  response: string
  evidence_refs: list[uuid] # Workstream ArtifactBinding IDs only
  created_at: timestamp
```

SubmissionFindingResponse invariants

- Immutable from creation.
- Required for every unresolved blocking finding when `version_number > 1`.
- `finding_id` must belong to the immediately prior review chain for the same task.
- It records the submitter's claim about how the finding was addressed; it does not mark the finding resolved.
- Only a later reviewer may record the canonical finding resolution.
- Every `evidence_ref` resolves to a verified Workstream ArtifactBinding owned by the same project and task chain.
- Raw provider CIDs, local paths, public URLs, and storage credentials are invalid evidence references at this boundary.

5.4 ReviewQueueEntry

`ReviewQueueEntry` is mutable routing infrastructure for exactly one submission version.

```
ReviewQueueEntry:
  id: uuid
  project_id: uuid
  task_id: uuid
  submission_version_id: uuid

  queue_state: enum [pending, leased, closed]
  routing_mode: enum [preferred, open]
  routing_reason: enum [first_submission, revision_return, admin_assignment]

  first_queued_at: timestamp
  available_since: timestamp

  preferred_reviewer_id: uuid | null
  preference_expires_at: timestamp | null

  active_lease_id: uuid | null

  closed_at: timestamp | null
  closed_reason: enum [review_recorded, task_closed, admin_cancelled] | null
```

ReviewQueueEntry creation

For a first submission version:

```
queue_state: pending
routing_mode: open
routing_reason: first_submission
preferred_reviewer_id: null
preference_expires_at: null
first_queued_at: now
available_since: now
```

For a revision after `needs_revision`:

```
queue_state: pending
routing_mode: preferred
routing_reason: revision_return
preferred_reviewer_id: prior_review.reviewers_id
preference_expires_at: now + project.review_preference_window
first_queued_at: now
available_since: now
```

For direct Admin routing to an eligible external or internal reviewer:

```
queue_state: pending
routing_mode: preferred
routing_reason: admin_assignment
preferred_reviewer_id: selected_reviewers_id
preference_expires_at: now + project.review_preference_window
first_queued_at: now
available_since: now
```

Admin assignment is still time-limited so work cannot remain hidden indefinitely in one reviewer backlog.

ReviewQueueEntry invariants

- Exactly one queue entry per submission version.
- `routing_mode = preferred` requires both a preferred reviewer and a future preference expiry.
- `routing_mode = open` requires preferred-reviewer fields to be null.
- `queue_state = leased` requires a valid `active_lease_id`.
- `queue_state IN (pending, closed)` requires `active_lease_id = null`.
- `queue_state = closed` requires `closed_at` and `closed_reason`.
- Closed entries cannot return to pending.
- Queue entries are never hard-deleted while audit retention applies.

5.5 ReviewLease

Every claim creates a separate `ReviewLease`. Lease attempts are not overwritten because they are future reviewer-reputation evidence.

```
ReviewLease:
id: uuid
review_queue_entry_id: uuid
reviewer_id: contributor_id
status: enum [active, consumed, released, expired, revoked]

claimed_at: timestamp
expires_at: timestamp
closed_at: timestamp | null
close_reason: enum [review_recorded, manual_release, lease_expired, grant_revoked,
admin_override] | null
```

ReviewLease invariants

- A reviewer may have at most one active lease globally in v0.1.
- A queue entry may have at most one active lease.
- A lease may be created only while the queue entry is pending.
- A lease may be consumed only through successful canonical Review creation.
- Terminal lease statuses never change.
- Lease timestamps use server/database time, not client time.
- Expired or revoked leases cannot authorize a Review.

5.6 Review

Review is the immutable canonical human decision against one submission version.

```
Review:
id: uuid
submission_version_id: uuid
review_queue_entry_id: uuid
review_lease_id: uuid

reviewer_id: contributor_id
decision: enum [accept, needs_revision, reject]
decision_reason: string | null

prior_review_id: uuid | null
reviewed_at: timestamp
lease_duration_actual: interval
```

Review invariants

- Immutable from creation.
- Unique constraint on `submission_version_id`.
- Reviewer must own the referenced active lease.
- Lease must belong to the referenced queue entry.
- Queue entry must belong to the referenced submission version.
- Reviewer grant must still be active at decision time.
- Reviewer cannot be the submission creator.
- Version 1 review has `prior_review_id = null`.
- Review of version N points to the Review of version N-1.

- `accept` and `reject` are terminal for the submission/review chain.
- `needs_revision` permits a later submission version if the submitter continues.
- `decision_reason` is required for `needs_revision` and `reject`.

The review and submission histories are two synchronized predecessor chains, not doubly linked lists. No forward pointer is required.

5.7 ReviewFinding

Findings are immutable records attached to a Review.

```
ReviewFinding:
id: uuid
review_id: uuid
description: string
severity: enum [blocking, advisory]
evidence_refs: list[uuid] # Workstream ArtifactBinding IDs only
created_at: timestamp
```

Decision and finding rules

- `accept`: findings are optional; any finding present must be advisory.
- `needs_revision`: findings are required and at least one finding must be blocking.
- `reject`: `decision_reason` is required; findings are optional but recommended.
- A finding is never edited to record its later resolution.
- Text-only findings are valid unless the task-locked review policy explicitly requires supporting finding evidence.
- When evidence is attached, every reference MUST already be verified, project/task-scoped, and visible to the acting reviewer through Workstream authorization.

5.8 FindingResolution

`FindingResolution` records the later reviewer's verdict about a prior finding without mutating that finding.

```
FindingResolution:
id: uuid
finding_id: uuid
submission_version_id: uuid
resolution_status: enum [resolved, unresolved, not_applicable]
rationale: string
recorded_by_review_id: uuid
recorded_at: timestamp
```

FindingResolution invariants

- Immutable from creation.
- Created only as part of reviewing a later submission version.
- Exactly one resolution per `finding_id` and reviewed `submission_version_id`.
- Every prior unresolved blocking finding must receive a resolution record in the next Review transaction.

- `accept` is invalid while any prior blocking finding is recorded as unresolved.
- A later `needs_revision` may leave prior findings unresolved and add new findings.

5.9 TaskAssignment extension

`TaskAssignment` is extended to preserve reject semantics.

```
TaskAssignment:
  id: uuid
  task_id: uuid
  contributor_id: uuid
  status: enum [active, completed, blocked]

  blocked_at: timestamp | null
  blocked_reason_review_id: uuid | null
```

TaskAssignment invariants

- `blocked` is scoped to one contributor-task relationship.
- A blocked contributor can never reclaim the same task.
- Blocking does not revoke a project role grant.
- Blocking does not affect other task assignments.
- `blocked_reason_review_id` points to the terminal reject Review.

5.10 Review artifact relationships

The review lifecycle reuses the existing Workstream Artifact Storage abstraction. It **MUST NOT** create a review-specific byte store.

The generic `ArtifactBinding` contract **MUST** support these review/revision roles:

```
submission_primary
submission_supporting_evidence
checker_input
checker_output
finding_response_evidence
review_finding_evidence
review_snapshot
```

The canonical Workstream binding contains or resolves at least:

```
ArtifactBindingReference:
id: uuid
project_id: uuid
task_id: uuid
owner_type: enum [submission_version, checker_run, finding_response, review_finding,
review]
owner_id: uuid
artifact_role: string
provider_ref: opaque string
content_digest: string
manifest_ref: opaque string | null
media_type: string
size_bytes: int
state: enum [pending, verified, unavailable, quarantined]
verified_at: timestamp | null
provenance_ref: opaque string | null
```

This is an integration view, not a second storage schema. If the Artifact Storage specification uses different internal names, its public port **MUST** still supply these semantics.

ReviewFindingEvidenceBinding

An immutable relation binds a canonical finding to already verified artifact evidence.

```
ReviewFindingEvidenceBinding:
id: uuid
review_finding_id: uuid
artifact_binding_id: uuid
created_at: timestamp
```

SubmissionFindingResponseEvidenceBinding

An immutable relation binds the submitter's response to evidence supporting that response.

```
SubmissionFindingResponseEvidenceBinding:
id: uuid
submission_finding_response_id: uuid
artifact_binding_id: uuid
created_at: timestamp
```

Binding invariants

- The referenced ArtifactBinding must be **verified** before it can be attached to a committed finding or finding response.
- The ArtifactBinding, finding/response, submission version, task, and project must resolve to one canonical project/task chain.
- One relation row is unique by its two referenced IDs.
- Relation rows are immutable and never hard-deleted while the Review chain is retained.
- Provider references are opaque to the review domain. Review services use **ArtifactStorePort**, never provider APIs directly.
- A provider CID or content digest proves byte identity; it does not prove project authorization, review correctness, or finding truth.

6. Project Review Policy and Artifact Storage Contract

6.1 Project review policy fields

The project policy context locked to the task MUST include:

```
ProjectReviewPolicy:
version: string
review_preference_window: interval
review_lease_duration: interval
max_active_review_leases_per_reviewer: int # fixed to 1 in v0.1
self_review_allowed: bool # fixed to false in v0.1
reject_policy: enum [close_task] # only v0.1 value
finding_evidence_requirement: enum [optional, required_for_blocking, required_for_all] #
default optional
```

These values are task-locked. A later project-policy change does not silently alter an already queued submission version.

`finding_evidence_requirement` determines whether a review finding must include verified ArtifactBinding references. It does not change the requirement that every finding include a concise description and severity.

- `optional`: findings may be text-only.
- `required_for_blocking`: every blocking finding has at least one verified evidence binding; advisory findings may be text-only.
- `required_for_all`: every finding has at least one verified evidence binding.

The v0.1 default is `optional`. The exact value is still frozen into the task's review policy context so a later project-policy change cannot alter an active review.

6.2 Canonical ownership boundary

```
Workstream owns
ActorProfile and authorization decisions
Task, SubmissionVersion and CheckerRun meaning
ReviewQueueEntry and ReviewLease state
Review, ReviewFinding and FindingResolution truth
ArtifactBinding meaning, lifecycle relationship and visibility
audit and outbox state

Artifact Storage owns
artifact bytes
provider content identity and manifests
integrity verification receipts
storage provenance
retention/pin operations
optional semantic/search projection
```

PostgreSQL remains the authority for whether a review exists and what it decided. Artifact Storage may preserve and index an immutable review snapshot, but that snapshot cannot create, alter, reverse, or delete a Review.

6.3 Required storage port

The review lifecycle depends only on a provider-independent `ArtifactStorePort` equivalent to:

```
class ArtifactStorePort:
    async def get_metadata(binding_ref, service_scope): ...
    async def retrieve_verified(binding_ref, service_scope): ...
    async def verify(binding_ref, expected_digest, service_scope): ...
    async def store(stream, source_ref, expected_digest, media_type,
        service_scope, idempotency_key): ...
    async def ensure_retained(binding_ref, retention_class, owner_ref,
        service_scope, idempotency_key): ...
    async def store_projection(stream, source_ref, version, expected_digest,
        service_scope, idempotency_key): ...
```

Implementations may combine calls internally, but they **MUST** preserve:

- exact content identity;
- idempotent storage operations;
- project/task scope enforcement;
- verified receipt fields;
- classified unavailable, not-found, integrity-mismatch, and scope-denied outcomes;
- no Workstream lifecycle mutations.

Successful mutating storage operations return a receipt equivalent to:

```
ArtifactReceipt:
    operation_id: string
    source_ref: string
    provider_ref: opaque string
    content_digest: string
    manifest_ref: opaque string | null
    size_bytes: int
    media_type: string
    verified: bool
    storage_state: string
    provenance_ref: opaque string | null
    retention_ref: opaque string | null
    occurred_at: timestamp
```

Workstream validates `source_ref`, digest, size, media type, scope and operation identity against its expected operation before changing an `ArtifactBinding` to `verified`.

6.4 Consuming artifacts for review

When the Reviewer opens Review Context, Workstream performs this sequence:

1. Verify Identity Issuer token and resolve current `ActorProfile`.
2. Require `submission.read_for_review` and `review.chain.read`.
3. Prove the actor owns the active `ReviewLease` for the submission version.
4. Resolve the locked task, guide, policy, submission, checker and prior-review chain.
5. Resolve canonical `ArtifactBinding` IDs from Workstream records.
6. Reject any cross-project, cross-task, unverified or quarantined binding.
7. Create a least-privilege service scope containing only the allowed project/task/binding actions.
8. Retrieve required bytes or metadata through `ArtifactStorePort`.
9. Verify the returned digest/manifest against the stored Workstream binding.
10. Record an attributable evidence-access receipt/audit reference.
11. Return the authorized Review Context.

The Reviewer receives the exact immutable submission and checker evidence governed by the task's locked context. The Reviewer does not choose an arbitrary provider CID or storage path.

Reading Review Context does not extend the lease, mutate a finding, or create a Review.

6.5 Submitting findings and finding evidence

The Reviewer submits the decision and structured findings to Workstream. They never submit a canonical finding directly to Artifact Storage.

When a finding includes an attachment or other artifact evidence:

1. The bytes enter through the existing Workstream artifact intake boundary.
2. Workstream supplies the canonical project/task scope, expected digest, media type, source reference, and idempotency key to `ArtifactStorePort`.
3. Artifact Storage returns a receipt containing the opaque provider reference, verified digest, manifest reference when supported, size, media type, provenance reference, and storage state.
4. Workstream creates or updates the corresponding generic `ArtifactBinding` to `verified` only after validating the receipt.
5. The decision request refers to the Workstream `ArtifactBinding` ID.
6. The decision transaction creates `ReviewFinding` and immutable `ReviewFindingEvidenceBinding` records together.

The same pattern applies to `SubmissionFindingResponse` evidence on revision.

An artifact uploaded for a decision that is never committed is an unbound artifact candidate. It may be removed later under the Artifact Storage orphan-retention policy, but no canonical Review or finding may be inferred from its existence.

6.6 Review transaction and storage projection

The canonical decision transaction MUST NOT perform a remote storage call while holding database locks.

All required finding-evidence `ArtifactBindings` are verified before the transaction begins. The transaction then:

```
creates Review
creates ReviewFinding records
creates ReviewFindingEvidenceBinding records
creates FindingResolution records
consumes ReviewLease
closes ReviewQueueEntry
applies Task and TaskAssignment effects
writes audit events
writes ReviewSnapshotProjectionRequested to the transactional outbox
commits
```

After commit, an outbox worker materializes an immutable review snapshot containing only the authorized projection fields:

- Review ID, decision, reason and reviewed time;
- project, task and submission-version references;

- locked policy version;
- finding IDs, descriptions, severities and evidence ArtifactBinding references;
- prior-review and finding-resolution references;
- canonical content digest and projection schema version.

The worker stores/indexes that snapshot through the configured Artifact Storage adapter and records the returned projection receipt in Workstream.

The projection idempotency identity is:

```
review:{review_id}:snapshot:{projection_schema_version}
```

The same identity and bytes return the same logical receipt. The same identity with different bytes is an integrity conflict and MUST alert operations.

6.7 Storage and integrity failure semantics

Failure	Review-lifecycle effect
Required artifact unavailable before claim	Claim is not created; return <code>review_evidence_unavailable</code>
Required artifact becomes unavailable during active lease	Decision is blocked; lease remains governed by its existing timer; record <code>ReviewEvidenceUnavailable</code>
Digest or manifest mismatch	Quarantine binding, block retrieval and decision, raise integrity alert
Finding attachment storage fails	No verified binding; Review decision requiring it cannot commit
Review snapshot projection fails after commit	Review remains canonical; outbox retries; projection status shows failed/pending
Semantic index unavailable	Review and artifact storage remain valid; retry projection without changing lifecycle state
Artifact provider response is uncertain	Use the existing idempotent outbox reconciliation contract; never guess success

An infrastructure failure MUST NOT be translated into `reject`, `needs_revision`, an unresolved finding, a checker failure, or contributor reputation evidence.

If a lease expires while a recorded evidence outage prevented completion, both `ReviewEvidenceUnavailable` and the lease transition remain in audit. Future reputation policy MUST be able to exclude infrastructure-caused delay; v0.1 does not assign a penalty.

6.8 Authorization between Workstream and Artifact Storage

Human authorization is completed by Workstream before any provider call.

```
Identity Issuer token
-> Workstream AuthorizationService
-> allowed ArtifactBinding set and actions
-> short-lived service request / configured service identity
-> Artifact Storage scope enforcement
```

The provider request scope includes only the necessary tenant/project, task, binding or provider references, action, expiry, and correlation identity. It MUST NOT include the human bearer token.

Artifact Storage scope enforcement is defense in depth. It is not a substitute for WS-AUTH-001, and a successful storage receipt is not an authorization decision.

6.9 Retention and chain reproducibility

- Every ArtifactBinding required by an active or retained review chain receives the configured retention/pin policy.
- A revision never changes the bytes bound to an earlier SubmissionVersion, ReviewFinding, or FindingResponse.
- A new artifact produces a new content digest and binding; prior bindings remain traversable.
- Retention release may occur only through the central Artifact Storage retention policy after all authoritative Workstream references permit it.
- Loss of a search index is recoverable from canonical Workstream records and retained artifact bytes.

6.10 Storage-adapter equivalence

The Local Storage adapter used in development/CI and the production Flow Node adapter MUST pass the same review-artifact conformance contract.

The provider may differ in CID, manifest, pinning, provenance, or indexing capabilities. Review authorization, binding visibility, digest verification, idempotency, unavailability handling, and Workstream lifecycle effects MUST remain identical.

7. Queue Views and Ordering

7.1 Preferred-reviewer backlog

A preferred backlog contains pending entries where:

```
routing_mode = preferred
AND preferred_reviewer_id = current_reviewer
AND preference_expires_at > now
```

This backlog explains why an individual reviewer may still have review work while the open project queue is empty.

Preferred entries are ordered by `first_queued_at` ASC.

7.2 Open FIFO pool

The open pool contains pending entries where:

```
routing_mode = open
```

Only entries for which the current actor is eligible are returned.

Open entries are ordered by:

```
first_queued_at ASC, id ASC
```

The ID tie-breaker makes ordering deterministic when timestamps match.

7.3 Reviewer next-work ordering

When Workstream selects the next claimable item for a reviewer:

1. Eligible preferred-backlog entries for that reviewer, oldest first.
2. Eligible open-pool entries, oldest first.

The server MUST NOT return another claimable entry if the reviewer already holds an active lease.

7.4 FIFO fairness on requeue

`first_queued_at` is immutable. Lease expiry and release update `available_since` but do not reset queue age.

```
first_queued_at = original queue entry time
available_since = time entry most recently became claimable
```

This prevents a contributor from losing queue priority because a reviewer claimed and stalled.

8. Reviewer Eligibility

A reviewer is eligible to claim a queue entry only if all conditions are true:

1. Identity Issuer token is valid for Workstream.
2. `WS-AUTH-001` resolves an active `ActorProfile` and active identity link.
3. `AuthorizationService` allows `review.claim` for the canonical `ReviewQueueEntry` project.
4. The allowed decision is backed by an active `ProjectRoleGrant(reviewer|both)` for that project.
5. Contributor is not the creator of the submission version.
6. Contributor has no active review lease anywhere in Workstream.
7. Contributor is not suspended or otherwise restricted by current Workstream policy.
8. Queue entry is pending.
9. If preference is active, contributor is the preferred reviewer.
10. No canonical Review already exists for the submission version.
11. All artifacts required to begin review have verified bindings and are currently retrievable.

Eligibility is checked inside the claim transaction and the applicable authority and resource guards are checked again inside the decision transaction. Artifact visibility and integrity are rechecked when `ReviewContext` is read and before decision commit.

9. State Machines

9.1 ReviewQueueEntry state machine


```

pending
-> leased authorized atomic claim
-> closed task/admin closure before review

leased
-> pending active lease released, expired, or revoked
-> closed canonical Review recorded

closed
-> terminal

```

Preference expiry does not change `queue_state`; it changes only routing:

```

pending + preferred
-> pending + open

```

9.2 ReviewLease state machine

```

active
-> consumed canonical Review recorded
-> released reviewer voluntarily releases
-> expired lease duration passes
-> revoked reviewer grant revoked or admin force-release

consumed | released | expired | revoked
-> terminal

```

9.3 Task review states

The queue state does not need to make the Task oscillate between pending and under-review. Queue and lease records provide that operational detail.

```

evaluation_pending
-> review_pending durable checker allows human review

review_pending
-> needs_revision Review decision
-> accepted Review decision
-> closed Review decision = reject

needs_revision
-> in_progress contributor continues revision work
-> closed contributor withdraws or admin closes under separate policy

```

When reject closes the task:

```

Task.status: closed
Task.closed_reason: review_rejected

```

10. Timer Semantics

10.1 Preference timer

The preference timer starts when a revision queue entry is created.

Purpose:

- preserve reviewer continuity;
- keep the reviewer familiar with the findings and prior evidence;
- avoid forcing a new reviewer to reconstruct the chain unnecessarily.

It does not reserve reviewer capacity and does not create a lease.

On preference expiry:

```
routing_mode -> open
preferred_reviewer_id -> null
preference_expires_at -> null
first_queued_at -> unchanged
available_since -> now
audit ReviewerPreferenceExpired
```

10.2 Lease timer

The lease timer starts only when an eligible reviewer successfully claims the queue entry.

Purpose:

- enforce one-review-at-a-time capacity;
- prevent indefinite work hoarding;
- allow automatic recovery when a reviewer stops working.

On lease expiry:

```
ReviewLease.status -> expired
ReviewLease.closed_at -> now
ReviewLease.close_reason -> lease_expired

ReviewQueueEntry.queue_state -> pending
ReviewQueueEntry.routing_mode -> open
ReviewQueueEntry.active_lease_id -> null
ReviewQueueEntry.preferred_fields -> null
ReviewQueueEntry.first_queued_at -> unchanged
ReviewQueueEntry.available_since -> now

audit ReviewerLeaseExpired
```

Lease expiry burns reviewer stickiness. The expired reviewer may later reclaim the entry from the open pool if still eligible, but receives no renewed preference.

11. Core Operations

11.1 Create queue entry after checker admission

Preconditions:

- task is `evaluation_pending`;

- durable checker run is completed;
- routing recommendation is `allow_review`;
- submission version has no queue entry.
- every required submission/checker `ArtifactBinding` is verified and retained;
- the consuming worker is the provisioned Workstream system actor processing the authoritative checker-completion event.

Atomic effect:

```
create ReviewQueueEntry
task.status -> review_pending
record ReviewQueueEntryCreated
record TaskStatusChanged(evaluation_pending -> review_pending)
```

Idempotency is required. Reprocessing the same checker completion MUST return or preserve the existing queue entry.

11.2 Claim review

Claim is an atomic compare-and-set operation.

Preconditions:

- `AuthorizationService` allows `review.claim` for the canonical queue-entry resource inside the claim transaction;
- actor passes all reviewer eligibility and resource-guard checks;
- queue entry is pending;
- no active lease exists for the queue entry;
- reviewer has no active lease;
- preferred-routing rules permit this reviewer.
- all artifacts required to begin review are verified and retrievable through `ArtifactStorePort`.

Atomic effect:

```
create ReviewLease(status=active)
ReviewQueueEntry.queue_state -> leased
ReviewQueueEntry.active_lease_id -> new lease ID
record ReviewerClaimedTask
```

The implementation MAY use `SELECT ... FOR UPDATE`, an atomic conditional `UPDATE`, or another database-safe compare-and-set strategy. It MUST NOT perform an unprotected read followed by a write.

11.3 Release claimed review

Only the active leased reviewer or authorized Admin may release.

- Reviewer release requires `review.release` and the owned-active-lease guard.
- Admin force-release requires `review.lease.force_release`, covered Operator scope, and a reason.

Reviewer release effect:

```
lease -> released/manual_release
queue entry -> pending/open
stickiness -> cleared
available_since -> now
first_queued_at -> unchanged
record ReviewerReleasedTask
```

Admin force-release effect:

```
lease -> revoked/admin_override
queue entry -> pending/open
stickiness -> cleared
record ReviewerLeaseForceReleased
record AdminOverrideApplied(reason required)
```

11.4 Decline preferred review before claim

The preferred reviewer may decline an unclaimed preferred entry without first creating a lease.

AuthorizationService MUST allow `review.decline_preference`; the lifecycle guard MUST then prove the actor is the current preferred reviewer and the entry is still pending/preferred.

Effect:

```
routing_mode -> open
preferred fields -> null
first_queued_at -> unchanged
available_since -> now
record ReviewerDeclinedPreference
```

This prevents work from waiting for the full preference window when the previous reviewer already knows they cannot continue.

11.5 Record review decision

Decision creation is one database transaction.

Preconditions:

- AuthorizationService allows `review.decision` for the canonical lease/review resource inside the decision transaction;
- reviewer owns the active lease;
- lease is not expired according to server/database time;
- reviewer grant remains active;
- reviewer is not the submission creator;
- queue entry is leased by the referenced lease;
- no Review exists for the submission version;
- decision/finding validation passes;
- required prior finding resolutions are supplied for a revision.
- all required submission, checker, finding-response and finding-evidence ArtifactBindings are verified, visible, and project/task-consistent.

Transaction effect:

```
create Review
create ReviewFinding records
create ReviewFindingEvidenceBinding records
create FindingResolution records
ReviewLease.status -> consumed
ReviewLease.close_reason -> review_recorded
ReviewQueueEntry.queue_state -> closed
ReviewQueueEntry.closed_reason -> review_recorded
ReviewQueueEntry.active_lease_id -> null
apply decision-specific Task and TaskAssignment changes
apply WS-CON-001 contribution/compensation changes when that downstream integration is
enabled
write all audit events
write ReviewSnapshotProjectionRequested to the transactional outbox
commit
```

No partial Review may survive if any decision-specific state change fails.

No remote Artifact Storage call occurs while the decision transaction holds database locks. Candidate finding evidence is ingested and verified before the transaction; the immutable Review snapshot is projected after commit.

12. Decision Contracts

12.1 Accept

Validation:

- no unresolved prior blocking finding;
- any new findings are advisory only;
- decision reason optional.
- every finding evidence reference satisfies the task-locked `finding_evidence_requirement` and resolves to a verified binding.

Effect:

```
Review(decision=accept) created
queue entry closed
lease consumed
Task.status -> accepted
TaskAssignment.status -> completed
emit ReviewAccepted
apply WS-CON-001: create reviewer completed_review ContributionRecord
apply WS-CON-001: create submitter accepted_submission ContributionRecord
```

The complete integrated decision transaction follows **WS-CON-001**: every valid Review creates one reviewer `completed_review` ContributionRecord; accept additionally creates one submitter `accepted_submission` ContributionRecord. `ContributionRecordRequested` is not emitted by the completed implementation. Compensation records and outbox instructions are created only as defined by **WS-CON-001**; Artifact Storage never creates them.

12.2 Needs revision

Validation:

- decision reason required;
- at least one finding required;
- at least one finding is blocking.
- every finding evidence reference satisfies the task-locked `finding_evidence_requirement` and resolves to a verified binding.

Effect:

```
Review(decision=needs_revision) created
findings created
queue entry closed
lease consumed
Task.status -> needs_revision
TaskAssignment remains active
emit ReviewNeedsRevision
apply WS-CON-001: create reviewer completed_review ContributionRecord
```

If the contributor continues:

```
Task.status -> in_progress
create SubmissionVersion(N+1)
create required SubmissionFindingResponse records
run existing finalization and checker spine
if allowed -> create preferred ReviewQueueEntry for prior reviewer
```

12.3 Reject

Validation:

- decision reason required.
- every supplied finding evidence reference satisfies the task-locked `finding_evidence_requirement` and resolves to a verified binding.

Atomic effect:

```
Review(decision=reject) created
queue entry closed
lease consumed

TaskAssignment.status -> blocked
TaskAssignment.blocked_at -> now
TaskAssignment.blocked_reason_review_id -> Review.id

Task.status -> closed
Task.closed_reason -> review_rejected

emit ReviewRejected
emit TaskAssignmentBlocked
emit TaskClosed
apply WS-CON-001: create reviewer completed_review ContributionRecord
```

Reject is terminal in v0.1:

- no new submission version may be created;
- the blocked submitter cannot reclaim the task;
- the contributor's project role grant remains unchanged;
- other project tasks remain unaffected.

Future `reopen_to_pool` behaviour is not implemented here.

13. Revision Routing

13.1 Same reviewer claims within preference window

```
R1 reviews v1 -> needs_revision
Submitter creates v2
v2 passes durable checker
queue entry v2 -> preferred to R1
R1 claims before preference expiry
R1 records Review(v2)
Review(v2).prior_review_id = Review(v1).id
```

13.2 Different reviewer after preference expiry

```
R1 reviews v1 -> needs_revision
Submitter creates v2
v2 enters R1 preferred backlog
R1 does not claim before preference expiry
entry becomes open with original queue age preserved
R2 claims
R2 receives complete prior chain
R2 records Review(v2)
Review(v2).prior_review_id = Review(v1).id
```

The prior chain is independent of reviewer identity.

13.3 Preferred reviewer becomes ineligible

Preference MUST be cleared immediately when Workstream detects that the preferred reviewer:

- lost or had their project reviewer grant revoked;
- became the submitter due to corrected attribution;
- was suspended;
- became otherwise disallowed by locked project policy.

Effect:

```
entry remains pending
routing_mode -> open
preferred_fields -> null
available_since -> now
first_queued_at -> unchanged
record ReviewerPreferenceInvalidated
```

13.4 Preferred reviewer has another active lease

The entry remains in that reviewer's preferred backlog until:

- they complete/release their active lease and claim it;
- they decline preference;
- the preference window expires;
- an Admin clears the preference.

Preferred routing does not bypass the one-active-lease capacity limit.

14. Grant Revocation During an Active Lease

Grant revocation and lease recovery **MUST** be coordinated.

WS-AUTH-001 owns the grant revocation transaction and immediately removes the permission candidate. It emits the authority invalidation signal consumed by this lifecycle. WS-REV-001 owns the resulting lease and queue transitions.

When a reviewer grant is revoked:

1. The grant becomes revoked immediately.
2. Any active lease held under that grant becomes **revoked**.
3. The queue entry returns to **pending/open**.
4. Preferred routing is cleared.
5. The reviewer can no longer record a canonical decision.
6. Historical lease and audit records remain.

The grant-revocation transaction **SHOULD** perform this recovery immediately. A background reconciliation worker **MUST** also detect and repair any missed active lease.

No serialized AuthorizationDecision, worker payload, MCP session, or previously opened Review Context preserves decision authority after revocation.

15. API Contract

Endpoint names may be adapted to existing router conventions, but capability boundaries and error semantics are locked.

Every protected endpoint **MUST**:

1. verify the Identity Issuer bearer token;
2. build the current **AuthorizationContext** through WS-AUTH-001;
3. load the resource and derive its canonical project/ownership relationships;
4. require the permission specified in Section 4.6;

5. invoke one review/revision application command or query;
6. return the existing Workstream response/error envelope.

Routers MUST NOT query grant tables, reconstruct role unions, trust a request-body project ID, implement lease ownership, or call an Artifact Storage provider directly.

15.1 Contributor-role grants

```
POST /v1/projects/{project_id}/role-grants
GET /v1/projects/{project_id}/role-grants
POST /v1/projects/{project_id}/role-grants/{grant_id}/revoke
```

Create request:

```
{
  "contributor_id": "uuid",
  "role": "reviewer",
  "qualification_snapshot_ref": "uuid",
  "grant_method": "manual"
}
```

15.2 Reviewer queue

```
GET /v1/projects/{project_id}/reviews/current
GET /v1/projects/{project_id}/reviews/queue # admin/operator/audit inspection only
```

The contributor Reviewer response exposes only the actor's currently leased review, the next review offered for claim in the selected project, or no available review:

```
{
  "state": "available_to_claim",
  "queue_entry_id": "uuid",
  "project_id": "uuid",
  "routing_mode": "preferred",
  "lease": null
}
```

The Reviewer endpoint never returns the complete preferred backlog or open queue. The inspection endpoint is a separate administratively authorized capability and is not a reviewer work-selection surface.

15.3 Claim

```
POST /v1/reviews/queue/{queue_entry_id}/claim
```

Response:

```
{
  "queue_entry_id": "uuid",
  "lease_id": "uuid",
  "lease_status": "active",
  "claimed_at": "timestamp",
  "expires_at": "timestamp"
}
```

15.4 Release or decline

```
POST /v1/reviews/leases/{lease_id}/release
POST /v1/reviews/queue/{queue_entry_id}/decline-preference
```

15.5 Record decision

```
POST /v1/reviews/leases/{lease_id}/decision
```

Needs-revision request example:

```
{
  "decision": "needs_revision",
  "decision_reason": "Two blocking requirements remain unresolved.",
  "findings": [
    {
      "client_finding_id": "finding-1",
      "description": "Required verifier evidence is missing.",
      "severity": "blocking",
      "evidence_refs": ["workstream-artifact-binding-uuid"]
    }
  ],
  "prior_finding_resolutions": []
}
```

`evidence_refs` contains Workstream ArtifactBinding IDs already verified through the generic artifact intake boundary. Raw provider CIDs, filesystem paths, arbitrary URLs, and credentials are rejected.

Revision review example:

```
{
  "decision": "accept",
  "decision_reason": null,
  "findings": [],
  "prior_finding_resolutions": [
    {
      "finding_id": "uuid",
      "resolution_status": "resolved",
      "rationale": "Verifier evidence is present and valid."
    }
  ]
}
```

15.6 Read immutable chain

```
GET /v1/tasks/{task_id}/review-chain
GET /v1/submission-versions/{version_id}/review
```

The task chain response includes versions, reviews, findings, finding responses, resolutions, and relevant lease summaries in chronological order.

16. Error Contract

HTTP	Code	Condition
401	<code>authentication_required</code>	Missing Identity Issuer bearer token

HTTP	Code	Condition
401	invalid_token	Token verification fails
403	permission_not_granted	Current ActorProfile/grants do not supply the required permission
403	actor_not_active	ActorProfile is suspended or deactivated for this operation
403	identity_link_revoked	External identity link is no longer active
403	reviewer_grant_required	No active reviewer/both grant for project
403	self_review_forbidden	Reviewer is the submission creator
403	lease_not_owned	Decision/release actor does not own active lease
403	artifact_binding_not_visible	Actor is not authorized for the referenced Workstream ArtifactBinding
409	resource_project_mismatch	Resource ownership chain resolves across different projects
409	reviewer_preference_active	Another reviewer attempts claim during active preference window
409	reviewer_capacity_reached	Reviewer already holds an active lease
409	queue_entry_not_pending	Claim attempted on leased or closed entry
409	queue_entry_not_leased	Decision attempted when entry is not leased
409	lease_expired	Decision attempted after lease expiry
409	review_already_recorded	Review already exists for submission version
409	task_assignment_blocked	Submitter attempts to reclaim rejected task
409	preference_not_owned	Actor tries to decline someone else's preference
409	finding_evidence_not_verified	Referenced finding evidence has no verified ArtifactBinding
409	artifact_integrity_mismatch	Provider digest/manifest does not match the canonical binding
409	artifact_idempotency_conflict	Same storage operation identity is reused with different bytes or parameters
422	decision_reason_required	Missing reason for needs_revision or reject
422	findings_required	needs_revision contains no findings
422	blocking_finding_required	needs_revision has no blocking finding
422	blocking_finding_on_accept	accept includes a new blocking finding
422	finding_resolution_required	Prior blocking finding was not evaluated
422	unresolved_finding_on_accept	accept attempted with unresolved prior blocking finding
422	finding_evidence_required	Locked review policy requires evidence for one or more findings
503	review_evidence_unavailable	Required submission/checker artifact is temporarily unavailable; no review outcome is recorded
503	artifact_storage_unavailable	Artifact operation cannot currently be completed or reconciled safely

Errors MUST use the existing structured Workstream error envelope.

Authorization denials, domain conflicts, checker outcomes, and storage/infrastructure failures are distinct categories. The API MUST NOT collapse all four into `unauthorized_actor`, `review_failed`, or another generic error.

17. Database Constraints and Indexes

The implementation MUST enforce core invariants in the database, not only in service code.

Required constraints:

```

UNIQUE ReviewQueueEntry(submission_version_id)
UNIQUE Review(submission_version_id)
UNIQUE SubmissionVersion(task_id, version_number)
UNIQUE FindingResolution(finding_id, submission_version_id)
UNIQUE ReviewFindingEvidenceBinding(review_finding_id, artifact_binding_id)
UNIQUE SubmissionFindingResponseEvidenceBinding(submission_finding_response_id,
artifact_binding_id)

```

Required partial uniqueness:

```

UNIQUE ReviewLease(review_queue_entry_id) WHERE status = 'active'
UNIQUE ReviewLease(reviewer_id) WHERE status = 'active'

```

Required indexes:

```

ReviewQueueEntry(project_id, queue_state, routing_mode, first_queued_at)
ReviewQueueEntry(preferred_reviewer_id, queue_state, preference_expires_at)
ReviewLease(status, expires_at)
ProjectRoleGrant(project_id, contributor_id, status, role)
SubmissionVersion(task_id, version_number)
Review(prior_review_id)
ReviewFinding(review_id)
FindingResolution(finding_id, submission_version_id)
ReviewFindingEvidenceBinding(review_finding_id)
ReviewFindingEvidenceBinding(artifact_binding_id)
SubmissionFindingResponseEvidenceBinding(submission_finding_response_id)
SubmissionFindingResponseEvidenceBinding(artifact_binding_id)
TaskAssignment(task_id, contributor_id, status)

```

Check constraints SHOULD enforce state-field compatibility for queue entries and leases.

Foreign keys and service guards MUST prove that each artifact-evidence relation resolves through one task and project. The decision transaction must lock or otherwise stabilize the referenced ArtifactBinding rows so they cannot transition away from **verified** between validation and commit.

18. Concurrency and Transaction Requirements

18.1 Claim race

If two reviewers claim the same entry concurrently, exactly one succeeds. The other receives 409 `queue_entry_not_pending` or an equivalent conflict.

18.2 Capacity race

If one reviewer attempts to claim two entries concurrently, exactly one active lease may be created. The other receives 409 `reviewer_capacity_reached`.

18.3 Decision versus expiry race

The server evaluates lease validity using one transaction and database time.

- If decision commit locks the active lease before the expiry transition, and the database time is not past `expires_at`, the decision may succeed.

- If expiry wins or the database time is past `expires_at`, decision fails with `lease_expired`.

18.4 Decision idempotency

The decision endpoint MUST require or derive an idempotency key. Repeating the same request returns the existing Review. Reusing the same key with a different payload returns an idempotency mismatch.

18.5 Reject atomicity

Review creation, queue closure, lease consumption, assignment blocking, task closure, and audit writes happen in one transaction.

18.6 Needs-revision atomicity

Review, findings, prior finding resolutions, queue closure, lease consumption, task transition, and audit writes happen in one transaction.

18.7 Authorization versus mutation race

Claim, release, decline, override, and decision transactions load current actor/grant state through `AuthorizationService.require(..., uow=transaction_uow)` before their mutation commits.

- If grant revocation commits first, the lifecycle mutation is denied.
- If the lifecycle mutation obtains the relevant authorization/grant locks and commits first, the later revocation applies to subsequent operations and performs required lease recovery.
- A router-level authorization result is insufficient for this race.

18.8 Artifact verification versus decision race

Finding evidence is stored and verified before the decision transaction. The decision transaction then validates and stabilizes the canonical `ArtifactBinding` rows before creating `ReviewFindingEvidenceBinding` records.

If integrity status changes to unavailable or quarantined first, decision commit fails. If a verified immutable binding is committed to the Review first, later provider availability loss does not rewrite the Review; recovery and retention processes restore access while the historical digest/reference remains unchanged.

18.9 Artifact operation uncertainty

Artifact storage and projection work uses the existing durable outbox and idempotent reconciliation contract. Provider call attempts may be repeated; provider logical effects must remain at most one for the operation identity.

An uncertain provider outcome never causes Workstream to fabricate a receipt, mark a binding verified, duplicate a review snapshot, or create a Review.

18.10 Contribution integration atomicity

When `WS-CON-001` is enabled, the Review, reviewer contribution, decision-specific submitter contribution, compensation awards/instructions, task/assignment transitions, audit events and transactional outbox records commit or roll back together as defined there.

Review snapshot projection remains post-commit Artifact Storage work and is not part of that database transaction.

19. Background Processing and Recovery

19.1 Preference-expiry sweep

Periodically finds:

```
queue_state = pending
routing_mode = preferred
preference_expires_at < now
```

and opens the entries idempotently.

19.2 Lease-expiry sweep

Periodically finds:

```
ReviewLease.status = active
expires_at < now
no canonical Review exists
```

and returns each entry to the open pool idempotently.

19.3 Lazy recovery

Queue list, claim, and decision operations SHOULD also detect stale preference/lease state so correctness does not depend exclusively on the sweep schedule.

19.4 Orphan reconciliation

A reconciliation job detects:

- leased queue entry without an active lease;
- active lease whose queue entry does not reference it;
- consumed lease without a Review;
- closed `review_recorded` queue entry without a Review;
- revoked grant with an active lease;
- Review whose task/submission chain does not match.

It MUST alert operators and use explicit compensating events rather than silent history edits.

19.5 Review snapshot projection worker

Consumes `ReviewSnapshotProjectionRequested` from the transactional outbox.

The worker:

1. loads the committed canonical Review and related records;
2. runs as the provisioned Workstream system actor;
3. materializes the versioned deterministic snapshot;
4. calls `ArtifactStorePort.store_projection` with `review:{review_id}:snapshot:{schema_version}`;
5. validates the receipt and records projection status/reference;
6. retries classified transient failures;
7. dead-letters unresolved ambiguity for Operator reconciliation without changing the Review.

19.6 Artifact-reference reconciliation

A reconciliation job detects:

- `ReviewFindingEvidenceBinding` whose `ArtifactBinding` is not verified;
- cross-project or cross-task artifact relationships;
- missing retention/pin state for an active review chain;
- committed Review without its expected snapshot projection status record;
- projection receipt digest mismatch;
- provider receipt confirmed while the Workstream outbox remains uncertain;
- provider content unavailable for a retained canonical binding.

The job repairs only mutable delivery, retention and projection state. It never edits a committed Review, finding, resolution, or historical content digest.

20. Audit Events

The following event types are required.

Authority

- `ProjectRoleGrantIssued`
- `ProjectRoleGrantRevoked`
- `ProjectRoleQualificationSnapshotCreated`

Routing

- `ReviewQueueEntryCreated`
- `ReviewRoutedToPreferredReviewer`
- `ReviewerPreferenceExpired`
- `ReviewerPreferenceInvalidated`
- `ReviewerDeclinedPreference`
- `ReviewQueueEntryOpened`
- `ReviewQueueEntryClosed`

Lease

- ReviewerClaimedTask
- ReviewerReleasedTask
- ReviewerLeaseExpired
- ReviewerLeaseRevoked
- ReviewerLeaseConsumed
- ReviewerLeaseForceReleased

Decision and revision

- ReviewRecorded
- ReviewAccepted
- ReviewNeedsRevision
- ReviewRejected
- ReviewFindingCreated
- FindingResolutionRecorded
- SubmissionFindingResponseCreated

Authorization linkage

- ReviewAuthorizationAllowed for configured sensitive mutations, linked to the canonical AuthorizationDecision
- ReviewAuthorizationDenied for configured denial classes, linked to the canonical AuthorizationDecision

These are lifecycle audit links, not a duplicate authorization-decision store. WS-AUTH-001 remains authoritative for the decision record.

Artifact evidence and projection

- ReviewEvidenceAccessed
- ReviewEvidenceUnavailable
- ReviewEvidenceIntegrityMismatch
- ReviewFindingEvidenceBound
- SubmissionFindingResponseEvidenceBound
- ReviewSnapshotProjectionRequested
- ReviewSnapshotProjected
- ReviewSnapshotProjectionFailed
- ReviewSnapshotProjectionReconciled

Task and assignment

- TaskAssignmentCompleted
- TaskAssignmentBlocked

- TaskClosed
- AdminOverrideApplied

Every event records:

```
event_id: uuid
event_type: string
occurred_at: timestamp
actor_id: actor_id | system_actor_id
project_id: uuid
task_id: uuid | null
submission_version_id: uuid | null
review_queue_entry_id: uuid | null
review_lease_id: uuid | null
review_id: uuid | null
reason: string | null
policy_version: string
correlation_id: string
authorization_decision_id: uuid | null
artifact_binding_ids: list[uuid]
artifact_operation_ref: string | null
```

Lease-expiry and voluntary-release events remain separate because future reputation policy may treat them differently.

21. Notifications and Operational Views

Notification delivery is an adapter concern, but the lifecycle emits events for:

- preferred review routed to reviewer;
- preference nearing expiry;
- lease nearing expiry;
- lease expired;
- review needs revision;
- review accepted;
- review rejected;
- reviewer grant revoked while work is leased.

Reviewer view

Must show:

- for a selected approved project, exactly one of: current active lease, next review offered for claim, or no review available;
- current active lease and remaining time when leased;
- prior submission/review chain for claimed work;
- unresolved findings and submitter responses.
- authorized verified artifact references and classified evidence-availability state.

The contributor Reviewer view **MUST NOT** expose the complete preferred backlog, open queue, queue depth, or alternative work items for cherry-picking.

Admin view

Must show:

- open FIFO depth per project;
- preferred backlog per reviewer;
- oldest preferred entry age;
- active lease count;
- leases approaching expiry;
- expired and voluntarily released lease counts;
- entries reopened because reviewer preference became invalid;
- review turnaround distributions.

The system **MUST NOT** report the queue as empty when preferred reviewer backlogs still contain pending work.

22. Observability

Required metrics include:

```
review_queue_open_depth{project_id}
review_queue_preferred_depth{project_id}
review_queue_oldest_open_age_seconds{project_id}
review_preferred_oldest_age_seconds{project_id}
review_active_lease_count
review_lease_expired_total{project_id}
review_lease_released_total{project_id}
review_claim_conflict_total{reason}
review_decision_total{decision, project_id}
review_turnaround_seconds{project_id, decision}
review_revision_count{project_id}
review_preference_fallback_total{project_id}
review_authorization_decision_total{permission,result,denial_code}
review_evidence_retrieval_total{result}
review_evidence_unavailable_total{project_id}
review_evidence_integrity_mismatch_total{project_id}
review_finding_evidence_binding_total{project_id}
review_snapshot_projection_total{result}
review_snapshot_projection_lag_seconds
```

Per-reviewer metrics should be available in authorized operational views but should not create uncontrolled high-cardinality public metric labels.

Structured logs **MUST** exclude tokens, credentials, raw private artifact content, sensitive source paths, signed retrieval capabilities, and unrestricted finding text. Logs link the authorization decision, lifecycle correlation ID, Workstream ArtifactBinding ID, and provider operation reference without logging provider secrets.

23. Security Requirements

- Review authority is evaluated by [WS-AUTH-001](#) from current Workstream state, not token role claims.
- No self-review in v0.1.
- Admin privileges do not bypass reviewer qualification.
- Reviewer decision requires an owned, active, unexpired lease.
- Grant revocation invalidates decision authority immediately.
- Queue and chain reads are project-authorized.
- Reviewer can read only entries they may claim, entries preferred to them, their active lease, and permitted historical context.
- Admin override requires a reason and dedicated audit event.
- Immutable records are corrected only through new compensating records/events.
- All evidence references shown during review must use the task's locked guide, policy, checker, and artifact context.
- Human bearer tokens are never forwarded to Artifact Storage.
- Reviewers never supply raw provider paths/CIDs as authoritative evidence references.
- Artifact Storage receives only least-privilege service scope for the allowed project, task, bindings and actions.
- Artifact Storage search results are re-authorized by Workstream before disclosure.
- Required evidence is digest/manifest verified before display and before decision commit.
- Artifact unavailability or integrity failure cannot be converted into an adverse human review outcome.
- Review snapshot projection contains only approved project-scoped fields and does not become a second canonical Review database.

24. End-to-End Reference Sequences

24.1 First submission accepted

1. Finalized submission creates `SubmissionVersion(v1)`.
2. Required submission `ArtifactBindings` are verified and retained.
3. Durable checker completes with `allow_review` and verified checker evidence.
4. Open `ReviewQueueEntry(v1)` is created.
5. R1's token resolves to an active `ActorProfile`.
6. `AuthorizationService` allows `review.claim` for R1 and the canonical project entry.
7. `ArtifactStorePort` confirms required review evidence is retrievable.
8. Eligible Reviewer R1 claims atomically; `ReviewLease L1` becomes active.
9. `AuthorizationService` allows `submission.read_for_review` and `review.chain.read`.
10. Workstream retrieves and verifies the exact artifact versions for Review Context.
11. R1 submits accept before L1 expires.
12. `AuthorizationService` allows `review.decision` inside the transaction.
13. `Review(v1)` is created; L1 is consumed; queue entry closes.
14. Task becomes accepted; `TaskAssignment` becomes completed.
15. [WS-CON-001](#) creates reviewer and accepted-submitter contributions in the decision transaction.
16. `ReviewSnapshotProjectionRequested` is emitted transactionally.
17. Projection worker stores the immutable review snapshot asynchronously.

24.2 Needs revision and same reviewer returns

1. R1 reviews v1 as needs_revision with blocking findings.
2. Any finding evidence is already stored, verified and referenced by Workstream ArtifactBinding ID.
3. Review, findings, evidence bindings, reviewer contribution and needs_revision task state commit atomically under WS-CON-001 integration.
4. Review(v1) snapshot is projected asynchronously.
5. Submitter reads their authorized review chain and creates v2 through the same submission.create capability.
6. Submitter includes responses and verified response-evidence bindings for every blocking finding.
7. v2 passes the durable checker.
8. ReviewQueueEntry(v2) is preferred to R1.
9. R1 is offered v2 and claims within the preference window.
10. Workstream authorizes and retrieves the complete v1/v2 artifact and review chain.
11. R1 records finding resolutions and a new decision.
12. Review(v2).prior_review_id points to Review(v1).

24.3 Needs revision and another reviewer takes over

1. v2 is preferred to R1.
2. R1 does not claim before preference expiry.
3. Preference clears; first_queued_at remains unchanged.
4. v2 becomes visible in the open eligible FIFO pool.
5. R2 claims.
6. R2 receives the complete v1/v2 submission and review chain.
7. R2 records the next Review and finding resolutions.

24.4 Claimed review expires

1. R1 claims queue entry Q1 and receives L1.
2. L1 expires without Review creation.
3. L1 becomes expired.
4. Q1 returns to pending/open.
5. Q1 preserves first_queued_at and updates available_since.
6. ReviewerLeaseExpired is recorded.
7. R1 has no renewed preference but may later claim from the open pool.

24.5 Reject

1. R1 holds active lease L1.
2. R1 records reject with a required reason.
3. Review is created and L1 consumed.
4. Queue entry closes.
5. Submitter TaskAssignment becomes blocked.
6. Task closes with review_rejected.
7. Reviewer completed_review ContributionRecord is created under WS-CON-001 integration.
8. No new submission version may be created in v0.1.

24.6 Required evidence temporarily unavailable

1. R1 is offered queue entry Q1.
2. AuthorizationService allows review.claim.
3. ArtifactStorePort reports a required submission binding unavailable.
4. Workstream does not create ReviewLease.
5. Q1 remains pending with its original FIFO age.
6. Workstream returns review_evidence_unavailable.
7. ReviewEvidenceUnavailable is recorded and storage recovery is triggered.
8. No checker failure, finding, Review or adverse contributor state is created.
9. After verified availability is restored, Q1 can be offered again.

24.7 Needs revision with finding attachment

1. R1 owns active lease L1 and has loaded verified Review Context.
2. R1 prepares a blocking finding with an evidence attachment.
3. Attachment enters the generic Workstream artifact intake boundary.
4. ArtifactStorePort stores/verifies it idempotently and returns a receipt.
5. Workstream creates verified ArtifactBinding A1 in the same project/task scope.
6. R1 submits review.decision using A1.id in the finding evidence_refs.
7. AuthorizationService re-allows review.decision inside the transaction.
8. Workstream validates A1 and creates Review, ReviewFinding and ReviewFindingEvidenceBinding.
9. Queue/lease/task effects commit with audit and projection outbox records.
10. Artifact Storage receives the immutable review snapshot after commit.

25. Conformance Tests

25.1 Authority tests

- Every protected review/revision operation calls the centralized AuthorizationService with a registered permission.
- Resource project scope is derived from canonical database relationships, not caller input.
- Authorization allow does not bypass queue, lease, self-review, task-ban or artifact guards.
- Authorization denial causes no lifecycle or artifact mutation.
- Claim and decision revalidate authority inside their database transactions.
- Submitter-only grant cannot claim review work.
- Reviewer-only grant cannot claim submission task work.
- **both** may perform both functions but cannot self-review.
- Reviewer grant on Project A cannot claim Project B review work.
- Admin without contributor reviewer grant cannot review.
- External reviewer behaves identically after local contributor onboarding and grant.
- Revoked grant cannot claim.
- Grant revoked during lease prevents decision.
- Human bearer token is never forwarded to Artifact Storage.

25.2 Queue-routing tests

- First submission enters open pool.
- Revision enters prior reviewer's preferred backlog.

- Non-preferred reviewer receives `reviewer_preference_active` during the window.
- Preferred reviewer may claim during the window.
- Preference expiry opens the entry.
- Preference expiry preserves `first_queued_at`.
- Preferred reviewer decline opens the entry immediately.
- Admin assignment creates time-limited preferred routing.
- General queue may be empty while preferred backlog is non-empty.
- Reviewer next-work ordering chooses preferred backlog before open pool.
- Contributor Reviewer view returns only active lease, one next offered review, or none; it never exposes the full queue.

25.3 Lease tests

- Two concurrent reviewers cannot claim the same entry.
- One reviewer cannot hold two active leases.
- Reviewer release returns entry to pending/open.
- Lease expiry returns entry to pending/open.
- Lease expiry clears stickiness.
- Expired reviewer may later claim from open pool if eligible.
- Expired lease cannot record decision.
- Release and expiry emit distinct audit events.
- Decision and expiry race resolves deterministically using database time.

25.4 Review tests

- A second Review cannot be stored for the same submission version.
- Review references the correct queue entry and lease.
- `needs_revision` without findings fails.
- `needs_revision` without a blocking finding fails.
- `accept` with a new blocking finding fails.
- `reject` without a decision reason fails.
- Decision endpoint is idempotent.
- Same idempotency key with different payload fails.
- Review cannot be edited after creation.
- Under completed `WS-CON-001` integration, every decision creates exactly one reviewer `ContributionRecord`.
- Under completed `WS-CON-001` integration, `accept` additionally creates exactly one submitter `ContributionRecord`.
- Needs-revision and reject create no submitter `ContributionRecord`.

25.5 Revision tests

- Version numbers increase monotonically.

- v2 points to v1.
- Review(v2) points to Review(v1) regardless of reviewer identity.
- Revised submission requires a response to each unresolved blocking finding.
- Original finding remains unchanged.
- Resolution is stored as a new FindingResolution.
- Accept fails if a prior blocking finding remains unresolved.
- Complete history is traversable from latest version backward.

25.6 Reject tests

- Reject closes the queue entry.
- Reject consumes the lease.
- Reject blocks the submitter's TaskAssignment.
- Blocked submitter cannot reclaim the task.
- ProjectRoleGrant remains active.
- Other task assignments remain unaffected.
- Task closes with `review_rejected`.
- No later submission version may be created.

25.7 Recovery tests

- Preference sweep is idempotent.
- Lease sweep is idempotent.
- Lazy access repairs stale expired preference/lease state.
- Reconciliation detects leased entry without active lease.
- Reconciliation detects consumed lease without Review.
- Reconciliation never silently deletes or rewrites immutable history.

25.8 Artifact evidence tests

- Reviewer can retrieve only ArtifactBindings belonging to the submission under their active lease.
- Cross-project and cross-task binding references fail closed.
- Required submission and checker artifacts are digest/manifest verified before display.
- Raw provider CID, path or arbitrary URL is rejected as a finding evidence reference.
- Text-only finding succeeds when locked policy makes evidence optional.
- Finding without evidence fails when locked policy requires it.
- Finding evidence must be verified before Review commit.
- ReviewFindingEvidenceBinding is immutable and project/task-consistent.
- SubmissionFindingResponse evidence follows the same binding rules.
- Required evidence unavailable before claim creates no lease.
- Required evidence unavailable during lease creates no Review or adverse contributor outcome.
- Integrity mismatch quarantines the binding and blocks decision.

- Local Storage and Flow Node adapters pass the same port conformance suite.

25.9 Review snapshot projection tests

- Review transaction commits its outbox request atomically with the Review.
- No remote storage call occurs inside the Review transaction.
- Projection is deterministic for Review ID and schema version.
- Same projection operation and bytes are idempotent.
- Same operation identity with different bytes fails as an integrity conflict.
- Projection failure does not roll back or mutate the canonical Review.
- Projection retry eventually records one logical receipt.
- Dead-letter reconciliation never creates a second Review.
- Search result is re-authorized through Workstream before disclosure.

26. Implementation Delivery Order

Phase 0: Dependency conformance

- complete WS-AUTH-001 Definition of Done;
- register `review.queue.override` and `artifact_recovery.request` in WS-AUTH-001 if not already implemented;
- prove ActorProfile resolution and current-grant revocation behavior;
- freeze the provider-independent ArtifactStorePort and ArtifactBinding contract;
- run the Local Storage adapter contract suite;
- prove durable artifact outbox idempotency and uncertain-operation reconciliation.

Exit condition: review code can consume stable AuthorizationService and ArtifactStorePort interfaces without querying grants or importing a provider adapter directly.

Phase 1: Schema and invariants

- Project role qualification snapshot
- ProjectRoleGrant updates
- SubmissionVersion constraints
- SubmissionFindingResponse
- ReviewQueueEntry
- ReviewLease
- Review
- ReviewFinding
- ReviewFindingEvidenceBinding
- SubmissionFindingResponseEvidenceBinding
- FindingResolution

- TaskAssignment extension
- Required unique and partial indexes

Exit condition: migrations and model-level constraint tests pass.

Phase 2: Authority and queue creation

- consume WS-AUTH-001 permission decisions and canonical ResourceContext loaders
- Admin grant/revoke operations remain owned by WS-AUTH-001
- reviewer eligibility service with centralized authorization and artifact availability preflight
- checker-admission to queue-entry creation
- preferred/open routing fields
- reviewer/admin queue reads

Exit condition: authority and routing conformance tests pass.

Phase 3: Claims and timers

- atomic claim
- global one-active-lease enforcement
- reviewer release
- preferred decline
- preference-expiry sweep
- lease-expiry sweep
- lazy recovery

Exit condition: claim concurrency and timer tests pass.

Phase 4: Immutable decisions

- decision validation
- immutable Review and ReviewFinding writes
- verified finding-evidence binding validation and writes
- FindingResolution writes
- accept and needs-revision transitions
- decision idempotency
- WS-CON-001 atomic contribution/compensation integration when that implementation phase is enabled

Exit condition: review and revision tests pass.

Phase 5: Reject and operational hardening

- assignment blocking
- terminal task closure
- grant-revocation lease recovery

- Admin force-release
- reconciliation worker
- review snapshot projection worker
- artifact-reference/retention reconciliation
- authorization/artifact metrics, dashboards, and notifications

Exit condition: reject, recovery, security, and observability tests pass.

Phase 6: Live API drill

Run one HTTP-visible reference flow covering:

```
first submission
-> preferred/open routing
-> claim
-> needs_revision
-> revised submission
-> preferred reviewer expiry or takeover
-> accept
```

Run a separate terminal reject flow, one lease-expiry recovery flow, one grant-revocation-during-lease flow, one finding-evidence attachment flow, and one Artifact Storage unavailable/recovery flow.

Exit condition: privacy-safe evidence report and internal trust bundle are approved.

27. Definition of Done

The review lifecycle is complete only when:

- every human request resolves one current ActorProfile and consumes one registered AuthorizationService permission;
- claim, release, decline, override, chain read and decision use canonical resource scope and required guards;
- no router or review service queries grants or reconstructs authorization independently;
- reviewer authority is based on explicit current project grants;
- self-review is impossible in v0.1;
- one reviewer cannot hold multiple active leases;
- preferred reviewer backlogs and the open queue are separately visible;
- first submissions route open and revisions route back to the prior reviewer;
- preference and lease timers operate independently;
- FIFO age is preserved when reviewer delay causes requeue;
- every lease attempt remains auditable;
- Review, ReviewFinding, FindingResolution, and SubmissionVersion are immutable;
- revision replays every unresolved blocking finding;
- accept, needs_revision, and reject transitions are atomic;

- reject blocks only the submitter-task assignment and closes the task in v0.1;
- completed [WS-CON-001](#) integration records one reviewer contribution for every Review and one submitter contribution only on accept;
- reviewers consume only authorized, verified `ArtifactBindings` for the exact submission/checker chain;
- finding and finding-response evidence uses immutable Workstream `ArtifactBinding` relations;
- human tokens never cross into Artifact Storage;
- storage outage or integrity failure blocks judgment without creating an adverse human outcome;
- committed Reviews project asynchronously and idempotently without making Artifact Storage canonical;
- Local Storage and Flow Node adapters satisfy the same review-artifact contract;
- runtime state and the HTTP-visible audit trail agree;
- conformance tests and live drills prove normal, failure, timeout, and concurrency paths.

28. Explicitly Out of Scope

- Automated reviewer-grant issuance from reputation thresholds.
- Reputation scoring formulas.
- Skills/reputation-based automated queue ranking.
- Multiple simultaneous review leases per reviewer.
- Self-review policy.
- Reviewer bidding or marketplace allocation.
- `ContributionRecord`, compensation-policy, award, adapter and fulfillment behavior beyond the normative [WS-CON-001](#) integration boundary.
- `ReputationEvent` implementation.
- Reject reassignment through `reopen_to_pool`.
- Public or cross-organization reviewer federation.
- Changes to Identity Issuer internal architecture.
- Changes to Flow Node artifact-storage internals.
- A second review-specific artifact store.
- Direct Reviewer access to Artifact Storage provider APIs.
- Making Artifact Storage or a semantic index authoritative for review state.
- Treating review snapshot projection as part of the canonical Review transaction.

29. Future Additive Directions

The following may be added without changing the v0.1 authority and immutable-chain model:

- automated [ProjectRoleGrant](#) issuance from a versioned policy engine;
- reviewer capacity greater than one under project policy;

- routing scores using skill, reputation, turnaround, and conflict data;
- separate submitter and reviewer reputation dimensions;
- project-level reject policy `reopen_to_pool`;
- reviewer cooldown after lease expiry;
- review escalation and adjudication;
- cross-organization expert-reviewer onboarding;
- evaluation-sprint batching if Workstream later needs sprint-level ownership and reporting.

None of these future directions may silently rewrite existing grants, queue events, leases, submissions, reviews, findings, or resolutions.

30. Coding-Agent Handoff Rules

The coding agent MUST:

1. Treat this specification as the review-lifecycle authority.
2. Inspect existing Workstream status enums, models, services, repositories, routers, audit patterns, and error envelopes before editing.
3. Preserve the proven intake spine and extend it only after `review_pending`.
4. Consume `WS-AUTH-001` through `AuthorizationService` and registered permissions; do not read grant tables or duplicate resource guards in routers.
5. Implement database constraints before relying on service checks.
6. Keep routers thin; lifecycle rules belong in services/domain guards.
7. Use database transactions for claims and decisions.
8. Use the existing async worker boundary for sweeps and reconciliation.
9. Consume the provider-independent `ArtifactStorePort`; never import Local Storage or Flow Node provider internals into review domain services.
10. Resolve public artifact references through Workstream `ArtifactBinding` IDs; never trust raw provider CIDs, URLs or paths from the caller.
11. Keep remote storage calls outside claim/decision database lock transactions; use verified pre-bound evidence and transactional outbox projection.
12. Add focused unit, service, API, concurrency, authorization, artifact-adapter, integrity, idempotency, recovery, and failure-mode tests.
13. Do not invent contribution or compensation behavior inside review code; integrate only the atomic `WS-CON-001` contract when that downstream implementation is included, and keep reputation scoring outside this change.
14. Produce evidence showing exactly what changed, what passed, and what remains unimplemented.
15. Stop and request architecture confirmation if existing code requires changing a locked decision in this document.